

4 Merge sort

08 March 2025 18:11

A	B	C
2	5	2
8	9	8
15	12	
18	17	
elements	n elements	

Time Complexity = $O(m+n)$
of merge

Algo Merge(A, B, m, n)

$i = 1, j = 1, k = 1$

while($i \leq m$ & $j \leq n$)

{ if($A[i] < B[j]$)

{ $C[k++] = A[i++]$

}
else { $C[k++] = B[j++]$

}

for(; $i \leq m$; $i++$)

{ $C[k++] = A[i]$

}

for(; $j \leq n$; $j++$)

{ $C[k++] = B[j]$

}

If there are more than 2 list

A	B	C	D
4	3	8	2
6	5	10	4

4

2

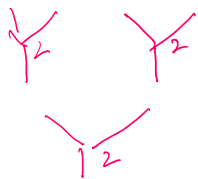
3

...

If 4 list are given we called
4 way merging

If n list
n way merging

If 2 list
2 way merging



Answer is same



2 way merge sort and merge sort

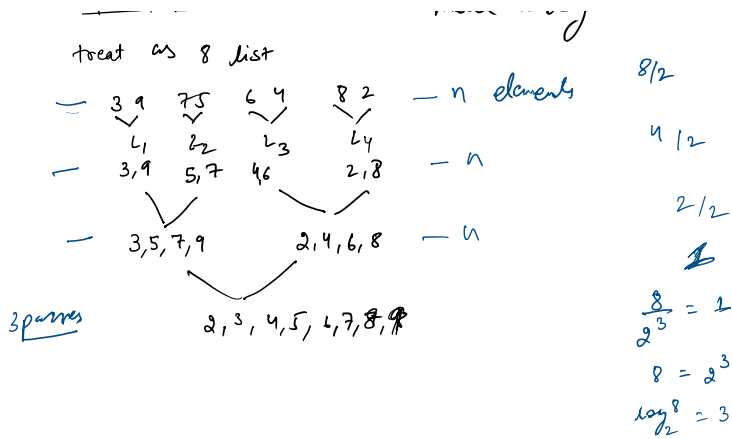
→ 2 way merge sort is iterative process with loop

→ merge sort is recursive procedure

2 way merge sort

1	2	3	4	5	6
A	1	3	7	5	2

for this we need some
more array



How many phases required = $\log n$
 Total time = $\Theta(n \log n)$

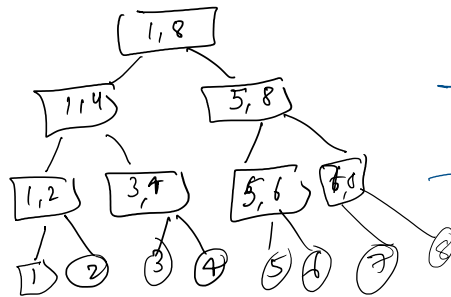
Merge Sort divide and conquer

A = [9, 3, 7, 5, 6, 4, 8, 2]

→ recursive algo

T(n) Algo mergesort(l, h)

① if (l < h)
 ② mid = (l+h)/2
 ③ merge(l, mid)
 ④ merge(mid+1, h)
 ⑤ merge(l, mid, h);
 n 8 list
 3



——— n
 ——— 8 list = n
 ——— 8 elements merge = n

3 phases

$\log 8 = \log n$

Total $\Theta = (n \log n)$

→ merging is happening in post order traversal

→ Recurrence relation

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + n & n>1 \end{cases}$$

$$a=2 \quad b=2 \quad f(n)=n$$

$$\log_b a = \log_2 2 = 1 \quad k = n^1 = 1$$

$$\log_b a = k$$

case 2

$$\Theta(n \log n)$$

2p = 35

pros & cons

① Large size list

② Linked list

③ External Sorting



① Extra space (not inplace sort) (in place of array not in linked list)

② NO small problem (if list is small then it takes more time $n < 15$)

its better to use Insertion Sort (is) because Insertion Sort is stable

Bubble Sort is also stable

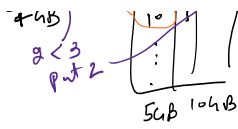
but merge sort and quick sort both are suitable for linked list

③ Recursive (may use stack in memory)

and stack depends on Tree

and tree is taking $\log n$

so its $O(\log n)$ space it takes in stack



exulany space = n
 totu space = $n + \log^n$
 $O(n)$

④ Stable Sort

ex 8, 6, 4, 8, 5, 9

after sort

3, 4, 5, 6, 8, 8, 9